Data Loading (ETL)

Q1. Data Understanding Identify all data quality issues present in the dataset that can cause problems during data loading

- Data quality issues are errors, inconsistencies, or missing values in a dataset that can cause problems during data loading, transformation, and analysis.
- The following data quality issues should be identified before loading the dataset into a tool such as Power BI, Excel, or a database.
- 1. Missing or NULL Values: Some rows may contain blank values in important columns, such as Product Name, Sales, Customer ID, or Region.
- 2. Duplicate Records: The same record may appear multiple times, which can lead to incorrect totals and calculations.
- 3. Incorrect Data Types: Columns may contain incorrect data types, such as sales values stored as text instead of numbers or dates stored as text.
- 4. Inconsistent Formatting: Dates, currency values, and text may follow different formats, making the data difficult to process consistently.
- 5. Extra Spaces: Leading or trailing spaces in text fields can cause problems when matching, filtering, or grouping data.
- 6. Invalid Values: The dataset may contain negative sales values, impossible dates, or values that violate business rules.
- 7. Inconsistent Category Names: The same category may be written differently, such as `North`, `north`, and `NORTH`, causing incorrect grouping.
- 8. Incorrect Column Names: Column names may contain spelling mistakes, extra spaces, or inconsistent naming conventions.
- 9. Special Characters: Unwanted symbols or unusual characters may cause errors during data import or affect text processing.
- 10. Inconsistent Data Structure: Some rows may contain extra or missing fields, or the dataset may have unexpected headers and blank rows.
- 11. Outliers: Extremely high or low values may indicate data-entry errors and can affect analysis.
- 12. Invalid or Missing Keys: Missing or duplicate primary keys, or foreign keys that do not match existing records, can cause problems when loading and relating tables.

Q3. Missing Value Analysis Which column(s) contain missing values? a) List the affected records b) Explain why loading these records without handling missing values is risky

Missing values are blank or NULL entries in a dataset where information is expected. They can affect data loading, calculations, and analysis.

a) Which columns contain missing values? List the affected records.

To identify the columns and records containing missing values, check the dataset for blank cells, NULL values, or empty strings.

For example, consider the following sample data:

Record ID	Product Name	Region	Sales
101	Laptop	North	50000
102	Mobile	—	25000
103	—	South	30000
104	Tablet	East	—

The missing values are:

- Region: Record 102
- Product Name: Record 103
- Sales: Record 104

Note: These are illustrative examples. The actual missing columns and records must be identified from your dataset.

b) Why is loading these records without handling missing values risky?

1. Incorrect calculations: Missing sales values can lead to incomplete or misleading totals and averages.
2. Data loading errors: Missing values in mandatory fields or primary keys may cause database constraints to fail.
3. Incorrect analysis: Missing region or product information can lead to inaccurate reports and visualizations.
4. Relationship issues: Missing customer or product IDs may prevent records from being linked correctly across tables.
5. Poor decision-making: Incomplete information can lead to unreliable business insights.

Q6. Load Readiness Decision Based on the dataset condition: a) Should this dataset be loaded directly into the database? (Yes/No) b) Justify your answer with at least three reasons

- ### a) Should this dataset be loaded directly into the database?

- Answer: No. The dataset should not be loaded directly into the database until the data quality issues have been identified and handled.
- **b) Justify your answer with at least three reasons.**
- 1. Missing Values: Blank or NULL values in important columns may cause errors or incomplete analysis.
- 2. Duplicate Records: Duplicate entries can lead to incorrect totals, double counting, and unreliable reports.
- 3. Inconsistent Data Types and Formats: Dates, numbers, and text stored in inconsistent formats can cause data loading errors and calculation problems.
- 4. Invalid Data: Incorrect or out-of-range values can produce misleading results and affect business decisions.
- 5. Data Integrity Issues: Missing IDs or invalid references can cause problems when creating relationships between database tables.

Q7. Pre-Load Validation Checklist List the exact pre-load validation checks you would perform on this dataset before loading.

- Before loading the dataset into a database, I would perform the following pre-load validation checks to ensure that the data is accurate, complete, consistent, and ready for loading.
- **Pre-Load Validation Checklist**

No.	Validation Check	What to Verify
1	Missing Values	Check for blank cells, NULL values, and empty strings in important columns.
2	Duplicate Records	Identify and remove unintended duplicate rows.
3	Data Types	Ensure columns have the correct data types, such as integer, decimal, text, and date.
4	Data Format	Verify consistent date formats, number formats, and currency formats.
5	Column Names	Check for duplicate column names, spelling errors, and extra spaces in headers.

6	Invalid Values	Identify negative sales values, impossible dates, and values that violate business rules.
7	Primary Key	Ensure primary key values are unique and not NULL.
8	Foreign Key	Verify that referenced IDs exist in the related tables.
9	Text Consistency	Remove unnecessary spaces and standardize capitalization and category names.
10	Column Structure	Confirm that every row has the correct number of columns and that headers are properly identified.
11	Outlier Detection	Investigate unusually high or low values that may indicate data-entry errors.
12	Business Rule Validation	Check that values follow business rules, such as valid quantities, prices, and sales amounts.
13	Encoding and Special Characters	Check for corrupted text, unwanted symbols, and encoding issues.
14	Row and Column Count	Record the total number of rows and columns before loading for comparison afterward.
15	Target Schema Compatibility	Confirm that source columns match the database table schema, including data types, lengths, and constraints.

- ## Example SQL Validation Queries
- 1. Check for missing values
- SELECT *
- FROM SalesData
- WHERE ProductName IS NULL

```
-    OR Region IS NULL
-    OR Sales IS NULL;
- 2. Check for duplicate IDs
- SELECT ProductID, COUNT(*) AS TotalCount
- FROM SalesData
- GROUP BY ProductID
- HAVING COUNT(*) > 1;
- 3. Check for invalid sales values
- SELECT *
- FROM SalesData
- WHERE Sales < 0;
```

Q8. Cleaning Strategy Describe the step-by-step cleaning actions required to make this dataset load-ready

To make the dataset ready for loading into a database, I would follow a step-by-step data cleaning process to improve data quality, consistency, and accuracy.

Step-by-Step Cleaning Actions

Step 1: Create a Backup of the Dataset

- Keep an original copy of the dataset.
- Perform cleaning on a separate working copy to avoid losing the original data.

Step 2: Handle Missing Values

- Identify blank cells, NULL values, and empty strings.
- Fill missing values using suitable methods, where appropriate.
- Remove records with missing critical information only when justified.
- Keep NULL values if the information is genuinely unknown and the database allows them.

Step 3: Remove Duplicate Records

- Identify exact and unintended duplicate records.
- Remove duplicates while preserving legitimate repeated transactions.
- Verify that primary key values are unique.

Step 4: Correct Data Types

- Convert numerical columns to integer or decimal types.
- Convert date columns into valid date formats.
- Ensure text columns contain text values.
- Resolve invalid values before conversion.

Data Loading (ETL)

Step 5: Standardize Data Formats

- Use a consistent date format, such as `YYYY-MM-DD`.
- Standardize currency and decimal formats.
- Ensure consistent capitalization in text fields and category names.

Step 6: Clean Text Data

- Remove unnecessary leading and trailing spaces.
- Remove unwanted characters where appropriate.
- Correct spelling errors and standardize category names.

Step 7: Validate Invalid Values and Outliers

- Identify negative sales, invalid quantities, impossible dates, and other values that violate business rules.
- Investigate unusually high or low values.
- Correct errors using verified information rather than making assumptions.

Step 8: Validate Keys and Relationships

- Check that primary keys are unique and not NULL.
- Verify that foreign keys match records in the related tables.
- Resolve missing or invalid references before loading.

Step 9: Standardize Column Names and Structure

- Remove extra spaces from column headers.
- Use clear, consistent column names.
- Remove unnecessary blank rows and repeated header rows.
- Ensure all records have the expected number of columns.

Step 10: Validate Against Business Rules

- Confirm that prices, quantities, and sales values follow the required rules.
- Check calculated fields, such as Total Sales = Quantity × Unit Price, if applicable.
- Verify that required fields contain valid values.

Step 11: Perform Final Validation

- Recheck missing values, duplicates, data types, and invalid records.
- Compare row counts before and after cleaning.
- Document all changes and confirm that no valid records were accidentally removed.

Step 12: Load the Cleaned Dataset

- Match the cleaned columns to the target database schema.
- Load the data into a staging table or test environment first.
- Verify row counts, constraints, and sample records before loading into the final table.

Q9. Loading Strategy Selection Assume this dataset represents daily sales data. a) Should a Full Load or Incremental Load be used? b) Justify your choice

a) Should a Full Load or Incremental Load be used?

Answer: Incremental Load should be used for daily sales data, especially when the database is updated regularly with new sales records.

b) Justify your choice.

1. Saves Time: Incremental loading processes only new or changed sales records instead of reloading the entire dataset every day.

2. Improves Performance: It reduces processing time and database workload, particularly when the sales dataset becomes large.

3. Avoids Duplicate Records: Using a unique transaction ID or another suitable key helps identify records that have already been loaded and prevents duplicate entries.

4. Supports Daily Updates: New sales transactions can be added regularly, while corrected or updated records can also be processed when required.

5. Uses Fewer Resources: Incremental loading generally consumes less memory, processing power, and network bandwidth than repeatedly loading the full dataset.

Example

Suppose the database already contains sales records for October 1–23, and new sales records arrive on October 24.

- Full Load: Reloads all sales records from October 1–24.
- Incremental Load: Loads only the new records for October 24 and processes any required updates.

Q10. BI Impact Scenario Assume this dataset was loaded without cleaning and connected to a BI dashboard. a) What incorrect results might appear in Total Sales KPI? b) Which records specifically would cause misleading insights? c) Why would BI tools not detect these issues automatically?

If the dataset is loaded into a Business Intelligence (BI) dashboard without cleaning, data quality issues can produce incorrect KPIs, misleading visualizations, and poor business decisions.

a) What incorrect results might appear in the Total Sales KPI?

The Total Sales KPI may display inaccurate values for the following reasons:

1. Duplicate Records: Sales transactions may be counted more than once, increasing total sales incorrectly.
2. Missing Sales Values: Incomplete records may cause total sales to be understated or exclude important transactions.
3. Invalid Sales Values: Negative or incorrect sales amounts may reduce or inflate the total.
4. Incorrect Data Types: Sales values stored as text may be excluded from calculations or cause conversion errors.
5. Inconsistent Categories: Sales may be grouped incorrectly by region or product, resulting in misleading comparisons.

b) Which records specifically would cause misleading insights?

The following types of records should be investigated:

Record Type	Example	Possible Impact
Duplicate transactions	The same transaction ID appears twice with the same sales amount.	Total Sales may be overstated.
Missing sales amount	A transaction has a blank Sales field.	Total Sales may be understated or incomplete.
Negative sales value	Sales = -5000 without a valid return or adjustment reason.	Total Sales may be understated.
Invalid sales value	Sales = 99999999 due to a data-entry error.	Total Sales may be overstated.
Sales stored as text	Sales = `"5000"` instead of a numeric value.	The value may not aggregate correctly.
Inconsistent region	Region is entered as `North`, `north`, and `NORTH`.	Regional sales may be split across categories.

Missing product name	A sales transaction has no Product Name.	Product-level analysis may be incomplete.

These are illustrative examples; the exact problematic records depend on the actual dataset.

c) Why would BI tools not detect these issues automatically?

1. BI tools rely on source data: Tools such as Power BI and Tableau generally visualize the data they receive. They cannot automatically determine whether every value is factually correct.

2. Valid data types do not guarantee accuracy: A sales amount can be stored as a valid number but still be incorrect.

3. Duplicates may be legitimate: BI tools cannot always distinguish accidental duplicate transactions from valid repeated purchases without business rules or unique identifiers.

4. Business rules are not always defined: A BI tool may not know whether negative sales represent an error, a refund, or a legitimate adjustment.

5. Missing values may be allowed: Blank values are often accepted and may simply be ignored in calculations, which can hide incomplete data.